Digital design phase project

Modeling & FPGA Implementation of CORDIC Algorithm

Name :Nada Yehia Elsayed

Table of Contents

1. Introduction 3
2. Objective 4
3. Cordic algorithm explanation 5
4. Matlab Modeling 7
   4.1. Matlab modeling process 7
   4.2. Matlab results 9
   4.3. Matlab results analysis 9
      1. Accuracy of CORDIC vs Reference 9
      2. Checkpoints on the Unit Circle 9
      3. Quadrant Handling 10
      4. Random Angles 10
      5. Decimal angles 10
      6. Angles at the maximum and minimum values of the fixed point format 10
      7. Fixed-Point Q2.14 Values 11
5. Simulation and verification 12
   5.1. Simulation results 13
5.2. Simulation wave forms 14
6. Synthesis results 15
   6.1. Synthesis summary 15
   6.2. Timing summary after synthesis 17
   6.3. Implementation/Placement 17
   6.4. Timing summary after implementation 17
   6.5. Synthesis Report of Critical Setup 18
   6.6. Synthesis Report of Critical Hold Path 18

1. Introduction

The **CORDIC (Coordinate Rotation Digital Computer)** algorithm is a highly efficient method for calculating a wide range of mathematical functions such as **sine, cosine, square root, logarithms, and vector magnitudes**. Its key advantage lies in the fact that it **avoids the use of complex multipliers**, relying instead on **simple additions, subtractions, and bit-shift operations**. Bit-shifting corresponds to multiplication or division by powers of two, which makes the algorithm especially well-suited for hardware implementations.

The core principle of CORDIC is based on performing a sequence of small, pre-calculated rotations to gradually approach the desired angle or function result. With each iteration, the algorithm refines the approximation, making it highly scalable in terms of precision.

Because of its simplicity and efficiency, CORDIC has found widespread use in many areas of modern technology:

- **Digital Signal Processing (DSP):** For operations such as filtering, spectrum analysis, and signal transformations (e.g., FFT, DCT).
- **Embedded Systems:** In real-time applications like motor control, robotics, and navigation where hardware-friendly math operations are needed.
- **FPGA and ASIC Design:** Ideal for implementing trigonometric and transcendental functions in hardware while saving area and power compared to traditional multipliers.
- **Calculators and Early Computing Devices:** Many early calculators and processors relied on CORDIC for trigonometric functions due to its efficiency.

CORDIC provides a **fast, hardware-efficient, and scalable solution** for performing complex mathematical calculations, making it an essential tool in DSP, embedded systems, and modern chip design.

2. Objective

The main objective of this project is to **design, implement, and verify a CORDIC-based hardware module** for calculating sine and cosine functions, following a complete design flow from high-level modeling to FPGA deployment. The work aims to ensure functional correctness, numerical precision, and hardware efficiency using fixed-point arithmetic.

The specific objectives are:

1. **Develop a MATLAB Golden Model**
   - Create a MATLAB reference implementation of the CORDIC algorithm that precisely mirrors the hardware design.
   - Use **fixed-point arithmetic** with:
     - **Angles in Q12.20 format (32-bit signed fixed-point)** for accurate angle representation.
     - **X, Y, sine, and cosine values in Q2.14 format (16-bit signed fixed-point)** to match the hardware data path.
   - Validate the algorithm's correctness and generate test vectors for hardware verification.
2. **Implement the Verilog CORDIC Module**
   - Implement a synthesizable Verilog design of the CORDIC algorithm for sine and cosine calculation.
   - Include **quadrant correction logic** to correctly handle input angles outside the $\pm\pi/2$ range.
3. **Design a Self-Checking Testbench**
   - Create a testbench that applies a comprehensive set of angle inputs to the CORDIC module.
   - Automatically compare the hardware outputs with the MATLAB reference results within a **±4 LSB≈ 0.000244 tolerance**
   - Ensure correct initialization of inputs (x_start, y_start) and synchronization using control signals (start, done).
4. **FPGA Deployment and Validation**
   - Target the implementation on the Artix-7 xa7a15tcsg324-2L
   - Demonstrate the functional correctness of the design under real hardware conditions.
   - Evaluate resource utilization and performance to confirm the design's hardware efficiency.

3. Cordic algorithm explanation

Suppose that we have an efficient system that receives a vector and rotates it by an arbitrary angle θ. Choosing the origin as the center of rotation, we will get to the point $(x_1\ ,y_1)$ by rotating the point $(x_0\ ,y_0)$ by θ

$$x_R = x_{in}\cos(\theta) - y_{in}\ \sin(\theta)$$

$$y_R = x_{in}\sin(\theta) + y_{in}\ \cos(\theta)$$

Equation 1

Equation 1 can be simplified to:

$$\begin{bmatrix} x_R \\ y_R \end{bmatrix} = \cos(\theta) \begin{bmatrix} 1 & -\tan(\theta) \\ \tan(\theta) & 1 \end{bmatrix} \begin{bmatrix} x_{in} \\ y_{in} \end{bmatrix}$$

Equation 2

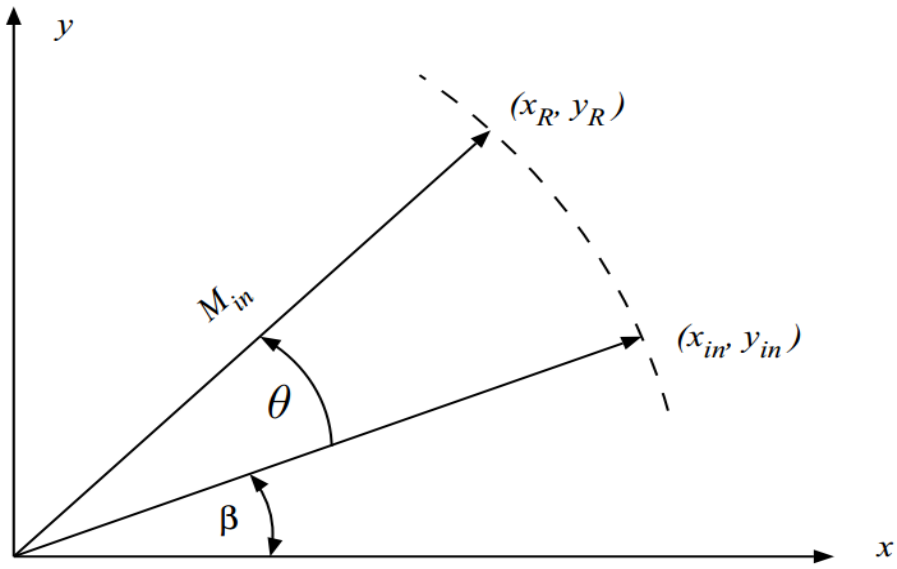


Figure 1. Rotating the input vector by θ

The CORDIC algorithm resorts to two fundamental ideas to achieve rotation without multiplication. The first fundamental idea is that rotating the input vector by an arbitrary angle θd is equal to rotating the vector by several smaller angles θi, i=0,1,…,n, provided θd= $\sum_{i=0}^{n}$ θi .

The second fundamental idea is that we can choose the small elementary angles in a way that tan(θi)= 2^{-i} for i=0,1,…,n. In this way, multiplication by tan(θi) can be achieved by a bitwise shift which is a much faster operation compared to

multiplication. The equation converges and by increasing n, we can increase the accuracy of calculations.

Each rotation mandates a scaling factor which appears in the final calculations. This means that it is possible to ignore the cos(θ) term of equation (2) and take the scaling factor into account at the end of the algorithm

$$K= \prod_{i=0}^{N-1} \cos(\theta i)$$

Therefore, omitting the scaling factor term from Equation 3, we obtain the following equations for the CORDIC algorithm:

$$x[i+1]=x[i]-\sigma i\ 2^{-i}\ y[i]$$

$$y[i+1]=y[i]+\sigma i 2^{-i} x[i]$$

Equation 4

where $\sigma i \in \{+1,-1\}$ determines the sign of the elementary angle

We choose the value of σi for each iteration of the algorithm by recording the angle of previous rotations and comparing the overall achieved rotation with the desired angle . If the desired rotation is larger than previously achieved rotations then σ=1 , so we need to rotate counter-clockwise (clockwise) in the next iteration. The described procedure is actually a negative feedback mechanism which computes the overall rotation and compares that to a reference value and chooses the new rotations in a way that minimizes the error signal ($\theta error=\theta d-\sum_{i=0}^{n} \theta i$).

$$x[i+1]=x[i]- \sigma i 2^{-i} y[i]$$

$$y[i+1]=y[i]+ \sigma i 2^{-i} x[i]$$

$$z[i+1]=z[i]-\sigma i \tan-1(2^{-i})$$

Equation 5

4. Matlab Modeling

4.1.Matlab modeling process

To verify the CORDIC algorithm before moving to hardware, a MATLAB model was developed. This script acts as a **golden reference**, ensuring the Verilog implementation is correct and that fixed-point formats are handled consistently.

1. **Fixed-Point Representation**
   - Angles are represented in **Q12.20 format** (32-bit signed fixed-point with 20 fractional bits).
   - Sine, cosine, and intermediate vector values are represented in **Q2.14 format** (16-bit signed fixed-point with 14 fractional bits).
   - This ensures the MATLAB model matches the hardware precision exactly.
2. **CORDIC Gain Compensation**
   - The CORDIC algorithm inherently scales the results by a constant gain factor K.
   - The script precomputes this gain for the chosen number of iterations (15 in this case) and uses it to correctly initialize the input vector.
3. **Lookup Table Generation**
   - A table of arctangent values (atan(2^-i)) is generated in Q12.20 format.
   - These values are written to a file (atan_table_verilog.txt) so that both MATLAB and Verilog implementations use the **exact same constants**, ensuring bit-accurate verification.
4. **Angle Preprocessing and Quadrant Handling**
   - Input angles are provided in degrees, then converted to radians and fixed-point format.
   - Angles are wrapped into the range [0,2π]and quadrant corrections are applied (flip signs of cosine/sine as needed). This matches the quadrant correction logic in Verilog.
5. **CORDIC Iterative Rotations**
   - The algorithm rotates the initial vector by successive angles from the lookup table, using only **shifts and additions/subtractions** (no multipliers).
   - After each iteration, the vector approaches the desired sine and cosine values.
6. **Result Comparison with Floating-Point Reference**
   - The final fixed-point sine and cosine values are scaled back to floating-point.
   - These are compared with MATLAB's built-in cos() and sin() functions for validation.

7. **Vector File Generation**

- For each test angle, the script saves the input angle (Q12.20) and the corresponding sine and cosine results (Q2.14) into a text file (cordic_vectors.txt).
- This file is directly used by the **Verilog testbench** to check the DUT outputs against MATLAB reference values.

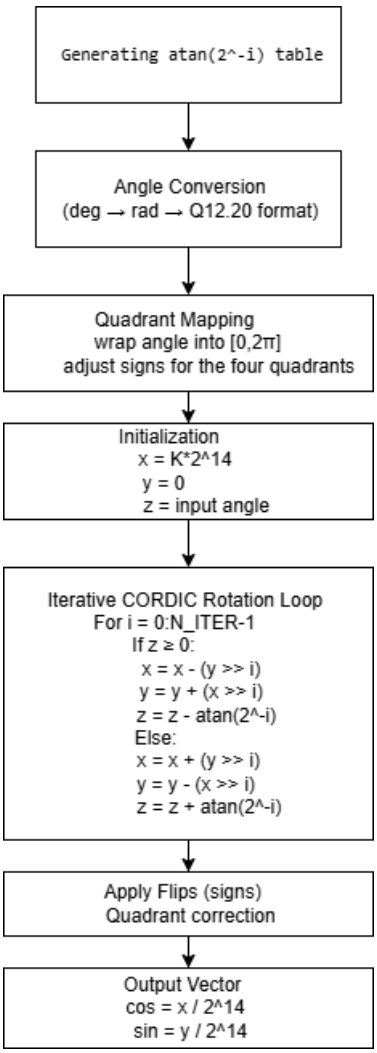


Figure 2. Matlab block diagram

4.2.Matlab results

Angle(deg)	Angle(Q12.20)	cosCORDIC	sinCORDIC	cosRef	sinRef	cos(Q2.14)	sin(Q2.14)
0	0	+0.999939	+0.000244	+1.000000	+0.000000	16383	4
90	1647099	+0.000244	+0.999939	+0.000000	+1.000000	4	16383
180	3294199	-0.999939	+0.000244	-1.000000	+0.000000	-16383	4
270	4941298	-0.000244	-0.999939	-0.000000	-1.000000	-4	-16383
360	6588397	+0.999939	+0.000244	+1.000000	-0.000000	16383	4
450	8235497	-0.000244	+0.999939	+0.000000	+1.000000	-4	16383
-90	-1647099	-0.000244	-0.999939	+0.000000	-1.000000	-4	-16383
-360	-6588397	+0.999939	+0.000244	+1.000000	+0.000000	16383	4
720	13176795	+0.999939	+0.000244	+1.000000	-0.000000	16383	4
520	9516574	-0.939514	+0.342102	-0.939693	+0.342020	-15393	5605
619	11328383	-0.190857	-0.981628	-0.190809	-0.981627	-3127	-16083
-223	-4081146	-0.731506	+0.681885	-0.731354	+0.681998	-11985	11172
627	11474792	-0.052368	-0.998596	-0.052336	-0.998630	-858	-16361
323	5911256	+0.798523	-0.601929	+0.798636	-0.601815	13083	-9862
-255	-4666781	-0.258789	+0.965942	-0.258819	+0.965926	-4240	15826
-59	-1079765	+0.515015	-0.857239	+0.515038	-0.857167	8438	-14045
231	4227555	-0.629211	-0.777100	-0.629320	-0.777146	-10309	-12732
675	12353245	+0.707092	-0.707153	+0.707107	-0.707107	11585	-11586
683	12499654	+0.798523	-0.601929	+0.798636	-0.601815	13083	-9862
-190	-3477210	-0.984741	+0.173767	-0.984808	+0.173648	-16134	2847
689	12609460	+0.857239	-0.515015	+0.857167	-0.515038	14045	-8438
674	12334944	+0.694641	-0.719360	+0.694658	-0.719340	11381	-11786
164	3001381	-0.961060	+0.275879	-0.961262	+0.275637	-15746	4520
505	9242057	-0.819092	+0.573547	-0.819152	+0.573576	-13420	9397
-207	-3788328	-0.891052	+0.453979	-0.891007	+0.453990	-14599	7438
95	1738605	-0.087280	+0.996033	-0.087156	+0.996195	-1430	16319
629	11511394	-0.017700	-0.999634	-0.017452	-0.999848	-290	-16378
496	9077347	-0.719360	+0.694641	-0.719340	+0.694658	-11786	11381
677	12389847	+0.731506	-0.681885	+0.731354	-0.681998	11985	-11172
5.550000e+01	1015711	+0.566467	+0.823975	+0.566406	+0.824126	9281	13500
6.988890e+02	12790440	+0.932739	-0.360168	+0.932884	-0.360176	15282	-5901
4.325670e+02	7916454	+0.299438	+0.953979	+0.299590	+0.954068	4906	15630
-1.173418e+05	-2147483647	+0.949646	+0.312927	+0.949734	+0.313058	15559	5127
1.173418e+05	2147483647	+0.949646	-0.312927	+0.949734	-0.313058	15559	-5127

vectors saved to cordic_vectors.txt

Figure 3. Matlab results

4.3.Matlab results analysis

1. Accuracy of CORDIC vs Reference

- The CORDIC results (cosCORDIC, sinCORDIC) are very close to the MATLAB reference values (cosRef, sinRef).
- The small difference (e.g., +0.999939 vs +1.000000) comes from:
  - Finite iterations (N_ITER = 15) → residual angle error.
  - Fixed-point quantization (Q12.20 angles, Q2.14 outputs) .

---

2. Checkpoints on the Unit Circle

- At **0°, 90°, 180°, 270°, 360°** the outputs match the expected unit circle values almost perfectly.
  Example:
  - At 0°, cos ≈ 0.999939 vs 1.0, sin ≈ 0.000244 vs 0.0.
  - At 180°, cos ≈ -0.999939 vs -1.0.

---

3. Quadrant Handling

- Angles outside ±360° (e.g., 450°, -360°, 720°) are correctly wrapped around into [0, 2π] range.
- Example:
  - 450° maps to 90° → sin ≈ +0.999939, cos ≈ 0.
  - -90° gives sin ≈ -0.999939, cos ≈ 0.

This shows the modulo-2π normalization and quadrant flip logic work correctly.

---

4. Random Angles

- For arbitrary test angles (e.g., 520°, 619°, -223°, 627°), the CORDIC output tracks the reference with very small deviation.
- Example:
  - At 619°, CORDIC gives (-0.190857, -0.981628) vs Ref (-0.190809, -0.981627)
  - At 323°, CORDIC gives (0.798523, -0.601929) vs Ref (0.798636, -0.601815).

This proves the iterative rotations converge properly across all quadrants.

---

5. Decimal angles

- At 55.5°, CORDIC gives (0.566467, 0.823975) vs Ref (0.566406, 0.824126)
- At 698.9°, CORDIC gives (0.932739, −0.360168) vs Ref (0.932884, −0.360176)

---

6. Angles at the maximum and minimum values of the fixed point format

- A signed 32-bit number ranges from -2^{31} to $2^{31}-1$
- But with Q12.20, the scaling factor is 2^{20}.
- Min=$-2^{31}/\ 2^{20}$=−2048 radians =−117341.8°
- Max=$(2^{31}-1)/\ 2^{20}$≈+2047.999999 radians=117341.8°

- At −117341.8°, CORDIC gives (0.949646, 0.312927) vs Ref (0.949972, 0.312335)
- At 117341.8°, CORDIC gives (0.949646, −0.312927) vs Ref (0.949972, −0.312335)

---

7. Fixed-Point Q2.14 Values

- The last columns (cos(Q2.14), sin(Q2.14)) show the **scaled 16-bit outputs**.
- For example:
  - At 0°, cos = 16383 (≈ 1.0 in Q2.14), sin = 4 (≈ 0.00024).
  - At 270°, cos = -4, sin = -16383.

These integer outputs are exactly what the Verilog hardware will produce.

Conclusion:
The results confirm that:

- The MATLAB golden model and the Verilog implementation (using Q12.20 angle, Q2.14 output) are consistent.
- Errors stay within **±0.000244 ,** which is the theoretical precision bound for this word length.
- The design handles angle normalization, quadrant correction, and iteration accuracy

5. Simulation and verification

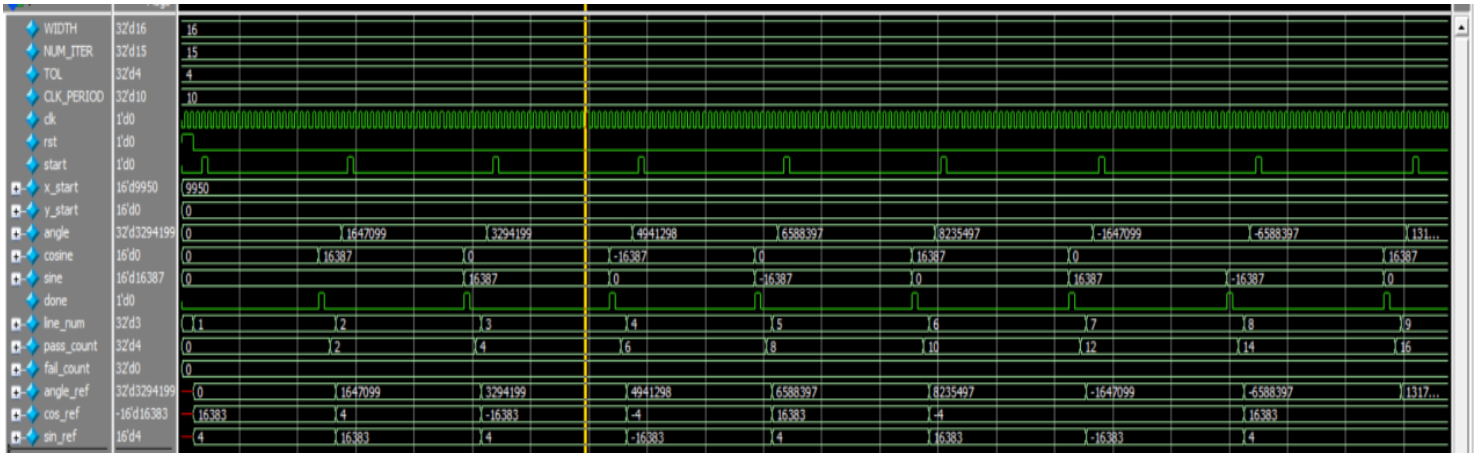
The Verilog testbench (CORDIC_TB.v) is designed to verify the correctness of the CORDIC module implementation for sine and cosine calculation. It provides a self-checking environment where the outputs of the hardware design are automatically compared against a golden reference generated by MATLAB.

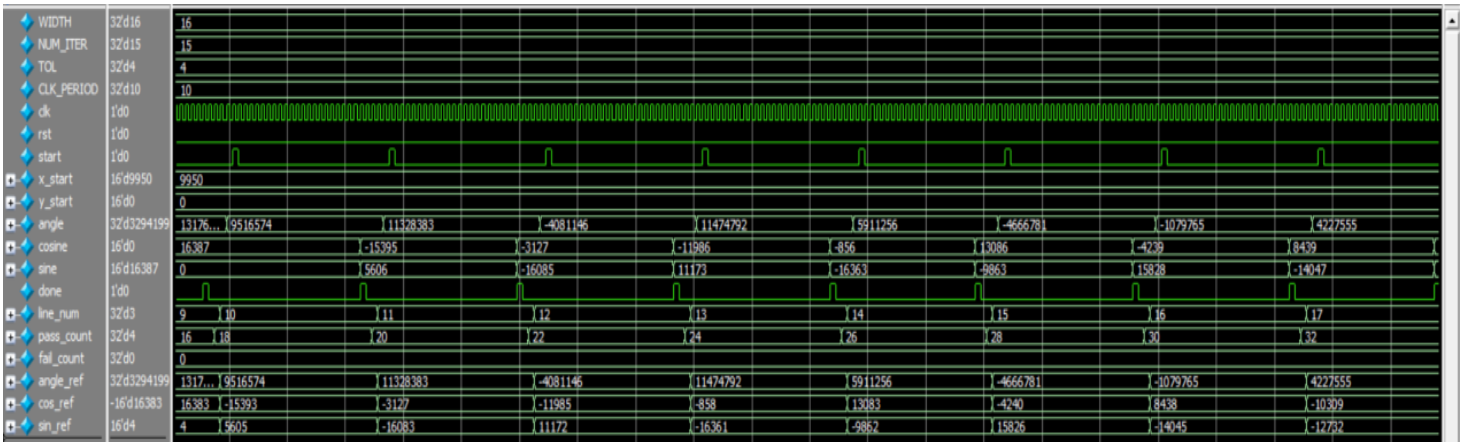
1. **DUT Instantiation:**
   The testbench instantiates the CORDIC module with configurable parameters (WIDTH=16, ITER=15) and connects all the input and output ports including clk, rst, start, angle, cosine, sine, and done.
2. **Clock and Reset Generation:**
   A system clock is generated with a period of 10 ns, and an initial reset sequence is applied to initialize the DUT before testing begins.
3. **Test Vectors Input (File-Driven Simulation):**
   - The testbench reads input angles and their corresponding expected cosine and sine values from the file **cordic_vectors.txt**, which was generated by the MATLAB golden model.
   - Each line in the file contains three values:
     - The test angle (Q12.20 fixed-point format).
     - The expected cosine (Q2.14 format).
     - The expected sine (Q2.14 format).
4. **Stimulus Application:**
   - For each test vector, the angle is applied to the DUT, and the start signal is pulsed for one clock cycle to initiate computation.
   - The testbench then waits for the DUT to assert the done signal, indicating the CORDIC iterations have completed.
5. **Self-Checking Mechanism:**
   - After completion, the DUT's outputs (cosine, sine) are compared against the expected values (cos_ref, sin_ref) read from the file.
   - A tolerance of **(±4 in Q2.14 format)** about **±0.000244 error** is allowed in the comparison to account for fixed-point rounding and CORDIC iteration error.
   - Results are reported as **PASS** or **FAIL** for each angle tested.
6. **Error Handling and Timeout:**
   - A timeout mechanism ensures the testbench does not stall if the done signal fails to assert within the expected number of cycles.
   - Malformed or incomplete lines in the vector file are ignored with a warning.
7. **Final Report:**
   At the end of the simulation, the testbench prints a **verification summary** showing:
   - Total number of test vectors processed.
   - Number of checks passed.
   - Number of checks failed.

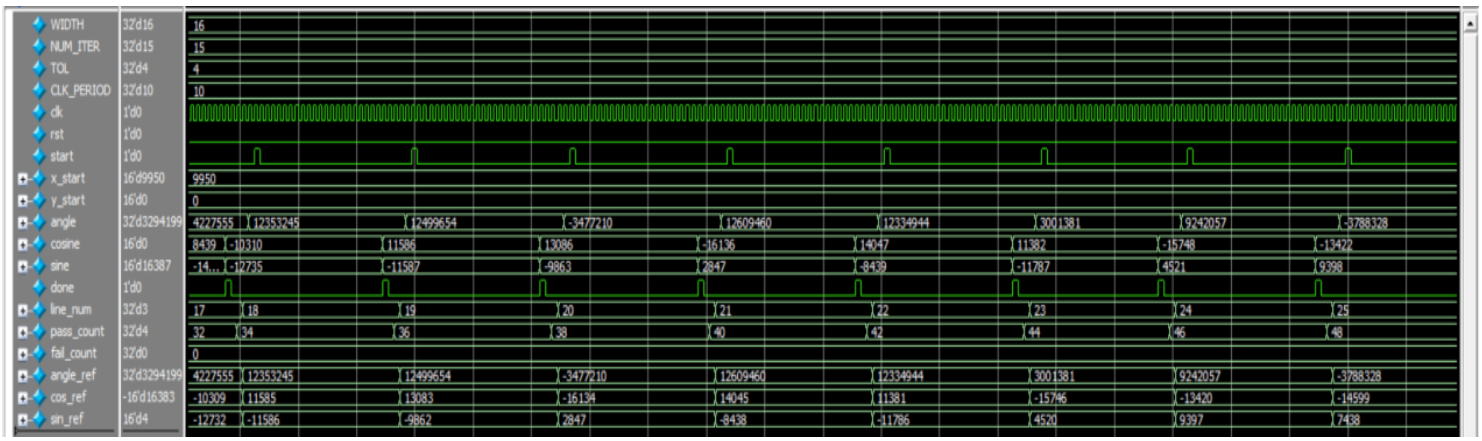
5.1.Simulation results

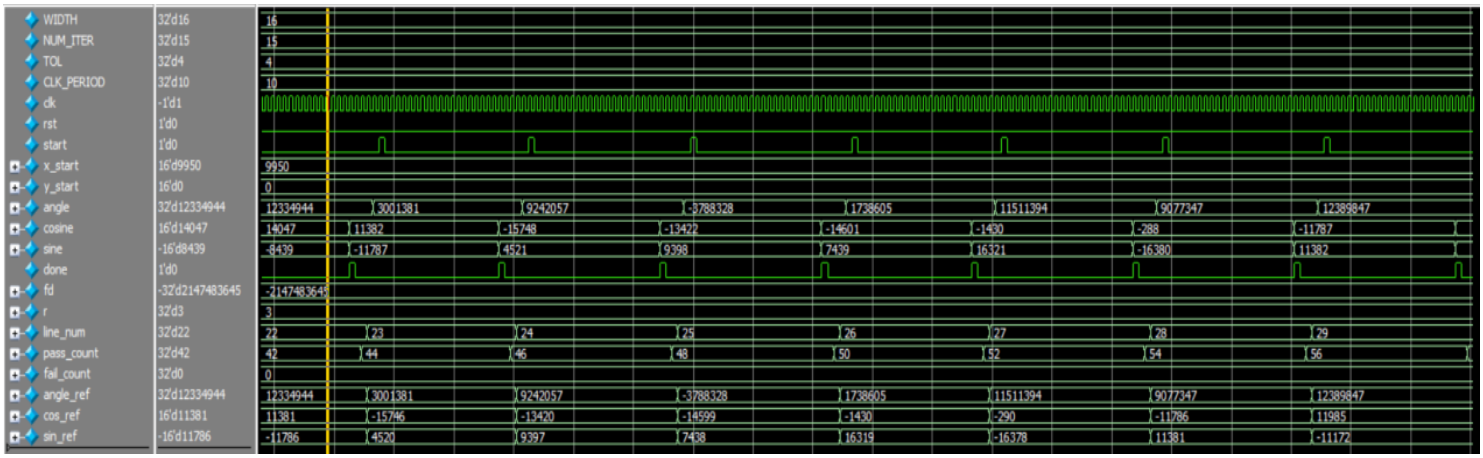
```
# --- Starting CORDIC Verification ---
#
# PASS [line 1]: angle=0 | cos=16387 (ref=16383)
# PASS [line 1]: angle=0 | sin=0 (ref=4)
# PASS [line 2]: angle=1647099 | cos=0 (ref=4)
# PASS [line 2]: angle=1647099 | sin=16387 (ref=16383)
# PASS [line 3]: angle=3294199 | cos=-16387 (ref=-16383)
# PASS [line 3]: angle=3294199 | sin=0 (ref=4)
# PASS [line 4]: angle=4941298 | cos=0 (ref=-4)
# PASS [line 4]: angle=4941298 | sin=-16387 (ref=-16383)
# PASS [line 5]: angle=6588397 | cos=16387 (ref=16383)
# PASS [line 5]: angle=6588397 | sin=0 (ref=4)
# PASS [line 6]: angle=8235497 | cos=0 (ref=-4)
# PASS [line 6]: angle=8235497 | sin=16387 (ref=16383)
# PASS [line 7]: angle=-1647099 | cos=0 (ref=-4)
# PASS [line 7]: angle=-1647099 | sin=-16387 (ref=-16383)
# PASS [line 8]: angle=-6588397 | cos=16387 (ref=16383)
# PASS [line 8]: angle=-6588397 | sin=0 (ref=4)
# PASS [line 9]: angle=13176795 | cos=16387 (ref=16383)
# PASS [line 9]: angle=13176795 | sin=0 (ref=4)
# PASS [line 10]: angle=9516574 | cos=-15395 (ref=-15393)
# PASS [line 10]: angle=9516574 | sin=5606 (ref=5605)
# PASS [line 11]: angle=11328383 | cos=-3127 (ref=-3127)
# PASS [line 11]: angle=11328383 | sin=-16085 (ref=-16083)
# PASS [line 12]: angle=-4081146 | cos=-11986 (ref=-11985)
# PASS [line 12]: angle=-4081146 | sin=11173 (ref=11172)
# PASS [line 13]: angle=11474792 | cos=-856 (ref=-858)
# PASS [line 13]: angle=11474792 | sin=-16363 (ref=-16361)
# PASS [line 14]: angle=5911256 | cos=13086 (ref=13083)
# PASS [line 14]: angle=5911256 | sin=-9863 (ref=-9862)
# PASS [line 15]: angle=-4666781 | cos=-4239 (ref=-4240)
# PASS [line 15]: angle=-4666781 | sin=15828 (ref=15826)
# PASS [line 16]: angle=-1079765 | cos=8439 (ref=8438)
# PASS [line 16]: angle=-1079765 | sin=-14047 (ref=-14045)
# PASS [line 17]: angle=4227555 | cos=-10310 (ref=-10309)
# PASS [line 17]: angle=4227555 | sin=-12735 (ref=-12732)
# PASS [line 18]: angle=12353245 | cos=11586 (ref=11585)
# PASS [line 18]: angle=12353245 | sin=-11587 (ref=-11586)
# PASS [line 19]: angle=12499654 | cos=13086 (ref=13083)
# PASS [line 19]: angle=12499654 | sin=-9863 (ref=-9862)
# PASS [line 20]: angle=-3477210 | cos=-16136 (ref=-16134)
# PASS [line 20]: angle=-3477210 | sin=2847 (ref=2847)
# PASS [line 21]: angle=12609460 | cos=14047 (ref=14045)
# PASS [line 21]: angle=12609460 | sin=-8439 (ref=-8438)
# PASS [line 22]: angle=12334944 | cos=11382 (ref=11381)
# PASS [line 22]: angle=12334944 | sin=-11787 (ref=-11786)
# PASS [line 23]: angle=3001381 | cos=-15748 (ref=-15746)
# PASS [line 23]: angle=3001381 | sin=4521 (ref=4520)
# PASS [line 24]: angle=9242057 | cos=-13422 (ref=-13420)
# PASS [line 24]: angle=9242057 | sin=9398 (ref=9397)
# PASS [line 25]: angle=-3788328 | cos=-14601 (ref=-14599)
# PASS [line 25]: angle=-3788328 | sin=7439 (ref=7438)
# PASS [line 26]: angle=1738605 | cos=-1430 (ref=-1430)
# PASS [line 26]: angle=1738605 | sin=16321 (ref=16319)
# PASS [line 27]: angle=11511394 | cos=-288 (ref=-290)
# PASS [line 27]: angle=11511394 | sin=-16380 (ref=-16378)
# PASS [line 28]: angle=9077347 | cos=-11787 (ref=-11786)
# PASS [line 28]: angle=9077347 | sin=11382 (ref=11381)
# PASS [line 29]: angle=12389847 | cos=11986 (ref=11985)
# PASS [line 29]: angle=12389847 | sin=-11173 (ref=-11172)
#
# --- Verification Summary ---
# Total vectors = 29
# Passed (checks) = 58
# Failed (checks) = 0
# ---------------------------------
```

5.2. Simulation wave forms









6. Synthesis results

6.1.Synthesis summary

```
Utilization Design Information

Table of Contents
-----------------
1. Slice Logic
1.1 Summary of Registers by Type
2. Memory
3. DSP
4. IO and GT Specific
5. Clocking
6. Specific Feature
7. Primitives
8. Black Boxes
9. Instantiated Netlists

1. Slice Logic
--------------

+-------------------------+------+-------+-----------+-------+
|        Site Type        | Used | Fixed | Available | Util% |
+-------------------------+------+-------+-----------+-------+
| Slice LUTs*             |  355 |     0 |     10400 |  3.41 |
|   LUT as Logic          |  355 |     0 |     10400 |  3.41 |
|   LUT as Memory         |    0 |     0 |      9600 |  0.00 |
| Slice Registers         |  176 |     0 |     20800 |  0.85 |
|   Register as Flip Flop |  176 |     0 |     20800 |  0.85 |
|   Register as Latch     |    0 |     0 |     20800 |  0.00 |
| F7 Muxes                |    0 |     0 |     16300 |  0.00 |
| F8 Muxes                |    0 |     0 |      8150 |  0.00 |
+-------------------------+------+-------+-----------+-------+

5. Clocking
-----------

+------------+------+-------+-----------+-------+
|  Site Type | Used | Fixed | Available | Util% |
+------------+------+-------+-----------+-------+
| BUFGCTRL   |    1 |     0 |        32 |  3.13 |
| BUFIO      |    0 |     0 |        20 |  0.00 |
| MMCME2_ADV |    0 |     0 |         5 |  0.00 |
| PLLE2_ADV  |    0 |     0 |         5 |  0.00 |
| BUFMRCE    |    0 |     0 |        10 |  0.00 |
| BUFHCE     |    0 |     0 |        72 |  0.00 |
| BUFR       |    0 |     0 |        20 |  0.00 |
+------------+------+-------+-----------+-------+
```

7. Primitives

Ref Name	Used	Functional Category
FDCE	176	Flop & Latch
LUT2	157	LUT
LUT3	130	LUT
LUT6	93	LUT
IBUF	67	IO
CARRY4	60	CarryLogic
LUT4	55	LUT
LUT1	47	LUT
OBUF	33	IO
LUT5	17	LUT
BUFG	1	Clock

4. IO and GT Specific

Site Type	Used	Fixed	Available	Util%
Bonded IOB	100	0	210	47.62
Bonded IPADs	0	0	2	0.00
PHY_CONTROL	0	0	5	0.00
PHASER_REF	0	0	5	0.00
OUT_FIFO	0	0	20	0.00
IN_FIFO	0	0	20	0.00
IDELAYCTRL	0	0	5	0.00
IBUFDS	0	0	202	0.00
PHASER_OUT/PHASER_OUT_PHY	0	0	20	0.00
PHASER_IN/PHASER_IN_PHY	0	0	20	0.00
IDELAYE2/IDELAYE2_FINEDELAY	0	0	250	0.00
ILOGIC	0	0	210	0.00
OLOGIC	0	0	210	0.00

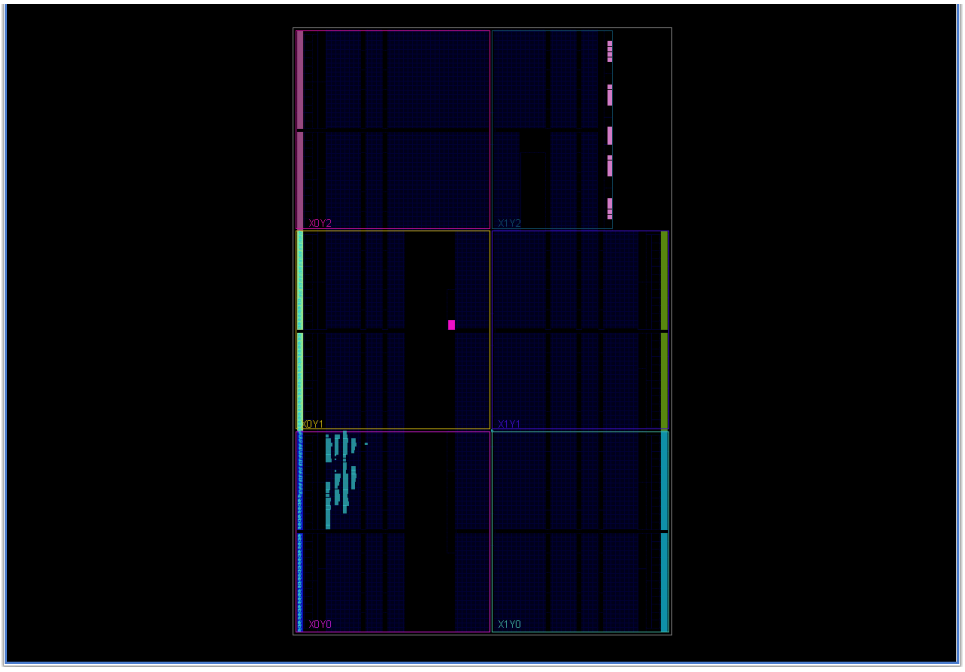
6.2.Timing summary after synthesis

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.005 ns	Worst Hold Slack (WHS):	0.152 ns	Worst Pulse Width Slack (WPWS):	
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	
Total Number of Endpoints:	315	Total Number of Endpoints:	315	Total Number of Endpoints:	

All user specified timing constraints are met.

6.3.Implementation/Placement



6.4.Timing summary after implementation

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	4.282 ns	Worst Hold Slack (WHS):	0.196 ns	Worst Pulse Width Slack (WPWS):	
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS	
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	
Total Number of Endpoints:	315	Total Number of Endpoints:	315	Total Number of Endpoints:	

6.5.Synthesis Report of Critical Setup

Intra-Clock Paths - clk - Setup

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 1	4.282	15	13	75	z_reg[2]/C	z_reg[28]/D	5.744	2.817	2.927	10.0	clk
Path 2	4.461	15	13	75	z_reg[2]/C	z_reg[31]/D	5.573	2.914	2.659	10.0	clk
Path 3	4.511	15	13	75	z_reg[2]/C	z_reg[29]/D	5.523	2.915	2.608	10.0	clk
Path 4	4.528	14	12	75	z_reg[2]/C	z_reg[24]/D	5.479	2.723	2.756	10.0	clk
Path 5	4.540	14	12	44	a_mod_reg[1]/C	a_tmp_reg[29]/D	5.455	2.633	2.822	10.0	clk
Path 6	4.545	14	12	44	a_mod_reg[1]/C	a_tmp_reg[31]/D	5.450	2.628	2.822	10.0	clk
Path 7	4.552	15	13	75	z_reg[2]/C	z_reg[30]/D	5.438	2.850	2.588	10.0	clk
Path 8	4.600	13	11	75	z_reg[2]/C	z_reg[20]/D	5.367	2.611	2.756	10.0	clk
Path 9	4.605	14	12	44	a_mod_reg[1]/C	a_tmp_reg[30]/D	5.390	2.568	2.822	10.0	clk
Path 10	4.607	14	12	75	z_reg[2]/C	z_reg[27]/D	5.362	2.797	2.565	10.0	clk

6.6.Synthesis Report of Critical Hold Path

Intra-Clock Paths - clk - Hold

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 11	0.196	0	1	4	x_reg[0]/C	cosine_reg[0]/D	0.283	0.141	0.142	0.0	clk
Path 12	0.211	1	1	7	x_reg[10]/C	cosine_reg[10]/D	0.347	0.186	0.161	0.0	clk
Path 13	0.245	1	1	1	a_tmp_reg[30]/C	z_reg[30]/D	0.368	0.187	0.181	0.0	clk
Path 14	0.251	1	1	1	a_tmp_reg[6]/C	z_reg[6]/D	0.398	0.190	0.208	0.0	clk
Path 15	0.252	1	1	1	a_tmp_reg[8]/C	z_reg[8]/D	0.398	0.192	0.206	0.0	clk
Path 16	0.257	1	1	1	a_tmp_reg[24]/C	z_reg[24]/D	0.402	0.183	0.219	0.0	clk
Path 17	0.258	1	1	1	a_tmp_reg[17]/C	z_reg[17]/D	0.394	0.186	0.208	0.0	clk
Path 18	0.260	1	1	1	a_tmp_reg[19]/C	z_reg[19]/D	0.396	0.186	0.210	0.0	clk
Path 19	0.260	1	1	1	a_tmp_reg[12]/C	z_reg[12]/D	0.390	0.186	0.204	0.0	clk
Path 20	0.263	1	1	2	load_reg/C	busy_reg/D	0.401	0.227	0.174	0.0	clk